Introduction to the *urbanperformance* Package

Contents

1 Introduction **1**
1.1 Indicators 1
1.2 Basic Data 2

2 The `urbanperformance` package **2**
2.1 Installation 2
2.2 Input data 3
2.3 Indicators calculation 4

1 Introduction

This vignette introduces the `urbanperformance` package, developed within the Urban Performance (UP) Tool framework. The UP Tool is designed to generate scenarios to project possible futures for specific study areas. The tool includes two main components: the Urban Performance Calculator and the Urban Performance Visualizer.

The `urbanperformance` package integrates a list of indicators aligned with the Sustainable Development Goals (SDGs). Included in the Urban Performance Calculator, these indicators assess a city's present and future performance by modeling the impacts of investment projects and policy interventions.

Through this package, users can access functions to evaluate multiple scenarios and spatial solutions, enabling rigorous testing of their effectiveness in advancing sustainable urban development.

1.1 Indicators

The `urbanperformance` package includes a list of indicators designated to assess city conditions across various scenarios. These indicators are numeric values that characterize the urban environment and identify critical issues, facilitating the evaluation, monitoring, and communication of a city's status. They are key for integrated urban planning, enabling users to model how an area currently addresses specific urban challenges.

The indicators included in the package are listed in the table below. Note that several of these metrics comprise sub-indicators to enhance the granularity of the analysis.

List of indicators	
1. Total population	11. Roads length
2. Urban footprint	12. Roads density
3. Population density	13. Intersection density
4. Land consumption	14. Cycle track density
5. Infill	15. Cycle proximity
6. Biodiversity land consumption	16. Public transport proximity
7. Agricultural land consumption	17. Amenities proximity
8. Green land consumption	18. Jobs proximity
9. Green land per capita	19. Hazard exposure
10. Land cover loss	

Similar to the Urban Performance Calculator, the `urbanperformance` package operationalizes the calculation methodology for these indicators. These methodologies are demonstrated in this document through a practical exercise. In essence, `urbanperformance` executes a series of geoprocessing tasks and calculations.

1.2 Basic Data

To calculate the indicators, the `urbanperformance` package requires two types of inputs: spatial data and numerical data.

1.2.1 Spatial Data

This comprises a set of georeferenced information layers, including:

Data	Description
Population	Spatial distribution of inhabitants
Urban Amenities location	Location of schools, hospitals, and basic services
Employment distribution	Spatial distribution of employment
Roads network	Location of roads, including categories
Land cover zones	Land cover areas and classification
Cycle tracks	Location of cycle tracks
Transport systems	Location of bus stops or transport routes
Hazard-prone areas	Location of hazard-prone areas in the city
Biodiversity areas	Delimitation of biodiversity zones
Green areas	Location of green areas in the city

The package supports various standard formats, including rasters (`.tif`) or vectors (`.shp`, `.kml`, `.geojson`).

Note: All spatial datasets must be cleaned and standardized prior to use within the package.

1.2.2 Numerical Data

In addition to spatial layers, numerical inputs are required. These include proximity parameters, land use classifications, and scalar values representing specific facts or assumptions about the urban environment.

To demonstrate the functionality of `urbanperformance`, this vignette utilizes pre-loaded dummy data based on a localized area in Cancún, Mexico. Users can use this dataset to explore the indicators and assess conditions for the example city.

2 The `urbanperformance` package

The exercise is integrated by four main steps to use the `urbanperformance` package:

1. Installation
2. Input data
3. Indicators calculation
4. Results analysis

2.1 Installation

The first step is to load the necessary packages to perform the complete example. Additionally, as can be consulted in the `urbanperformance` package documentation, the calculations included use functions from other packages to simplify processes.

```
# Load require libraries
library(urbanperformance)
```

```r
library(raster)
library(sf)
library(dplyr)
```

2.2 Input data

The second step is to load the city data. This data could be gathered from open data sources or local sources, depending on availability. For this exercise, a dummy dataset is included in the package to perform an example of application of the calculations.

The example dataset is already cleaned and standardized following a specific structure, described in the `urbanperformance` documentation. This dataset includes information about population, roads, land cover, cycle infrastructure, amenities location, hazard areas, and jobs proximity.

Dataset can be loaded as follows:

```r
# Load spatial data
data(aoi.cun)
data(roads.cun)
data(transport.cun)
data(cycle.cun)
data(amenities.cun)
data(land.cover.classes)

## Raster data
population_base <- system.file("extdata", "POP_2025.tif",
  package = "urbanperformance"
)
population_base <- raster::raster(population_base)

population_horizon <- system.file("extdata", "POP_2030.tif",
  package = "urbanperformance"
)
population_horizon <- raster::raster(population_horizon)

buildup_base <- system.file("extdata", "Build_up_2025.tif",
  package = "urbanperformance"
)
buildup_base <- raster::raster(buildup_base)

buildup_horizon <- system.file("extdata", "Build_up_2030.tif",
  package = "urbanperformance"
)
buildup_horizon <- raster::raster(buildup_horizon)

jobs_distribution <- system.file("extdata", "Jobs.tif",
  package = "urbanperformance"
)
jobs_distribution <- raster::raster(jobs_distribution)

hazard_zones <- system.file("extdata", "Hazard.tif",
  package = "urbanperformance"
)
hazard_zones <- raster::raster(hazard_zones)
```

```r
land_cover_18 <- system.file("extdata", "Land_cover_2018.tif",
  package = "urbanperformance"
)
land_cover_18 <- raster::raster(land_cover_18)

land_cover <- system.file("extdata", "Land_cover.tif",
  package = "urbanperformance"
)
land_cover <- raster::raster(land_cover)
```

The `rasterchecker()` function ensures that all spatial rasters share the same resolution, projection, and bounding box as the base population raster.

```r
## Verify urban extents
hazard_zones <- rasterchecker(hazard_zones, base = population_base)
buildup_base <- rasterchecker(buildup_base, base = population_base)
buildup_horizon <- rasterchecker(buildup_horizon, base = population_base)
jobs_distribution <- rasterchecker(jobs_distribution, base = population_base)
land_cover <- rasterchecker(land_cover, base = population_base)
```

2.3 Indicators calculation

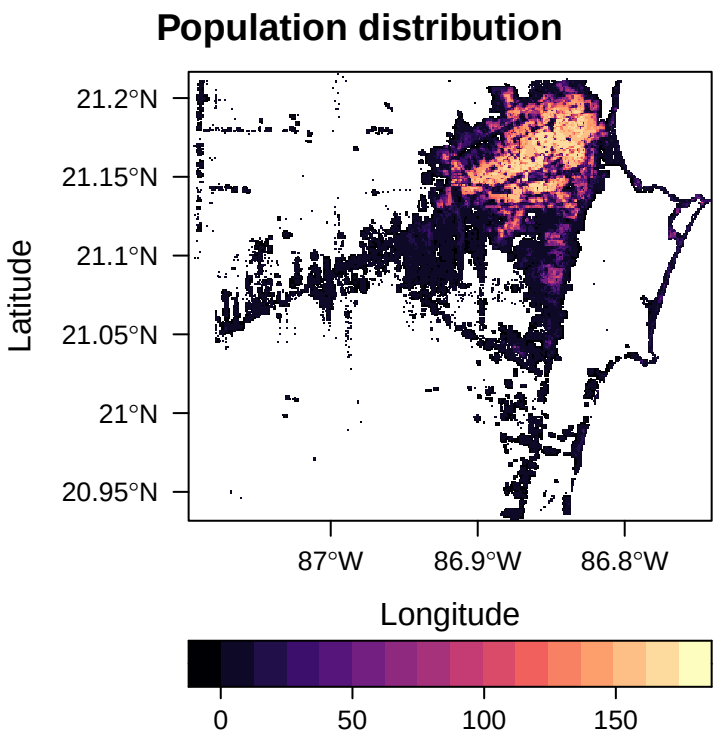
2.3.1 Total population

This function calculates the total population in the urban area by adding the population located in each pixel.

This metric serves as the fundamental baseline for the entire analysis. Accurate population counts are essential for normalizing other indicators (creating per-capita metrics) and for dimensioning the scale of urban needs, such as utility demand and service coverage.

```r
## Library for plotting rasters
library(rasterVis)
#> Loading required package: lattice

levelplot(population_base,
  margin = FALSE, colorkey = list(space = "bottom"),
  main = "Population distribution"
)
```



Calculate total population:

```
# library for plotting tables
library(kableExtra)
#>
#> Attaching package: 'kableExtra'
#> The following object is masked from 'package:dplyr':
#>
#>     group_rows

total_population <- tot_pop(population_base)

kable(total_population, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Total population	POP_2025	947982	inhabitants

2.3.2 Urban Footprint

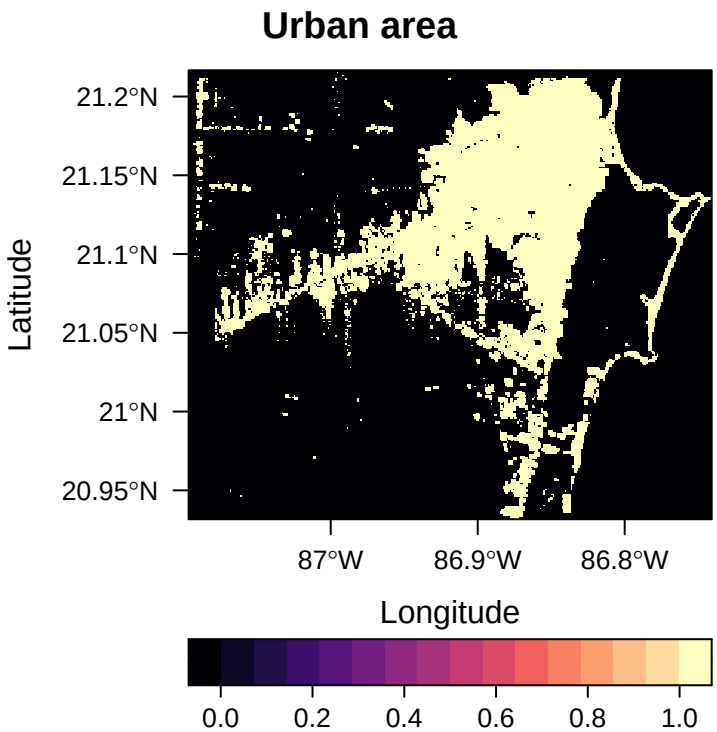
This function quantifies the total physical area occupied by artificial surfaces, built-up areas, and urban structures.

```
urban_footprint <- urban_footprint(buildup_base)

kable(urban_footprint, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Urban footprint	Build_up_2025	300.16	km2

```r
levelplot(buildup_base,
  margin = FALSE, colorkey = list(space = "bottom"),
  main = "Urban area"
)
```



2.3.3 Population density

Population density is the amount of inhabitants per built-up area. This indicator measures the intensity of land use by dividing the total population by the urban footprint area.

Density is a key driver of urban vitality and sustainability. Higher densities can support better public transport viability and local businesses, while very low densities may indicate inefficient land use. This indicator helps assess whether the city is meeting density targets required to support sustainable infrastructure.

```r
pop_dens <- population_density(population_base, buildup_base)

kable(pop_dens, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

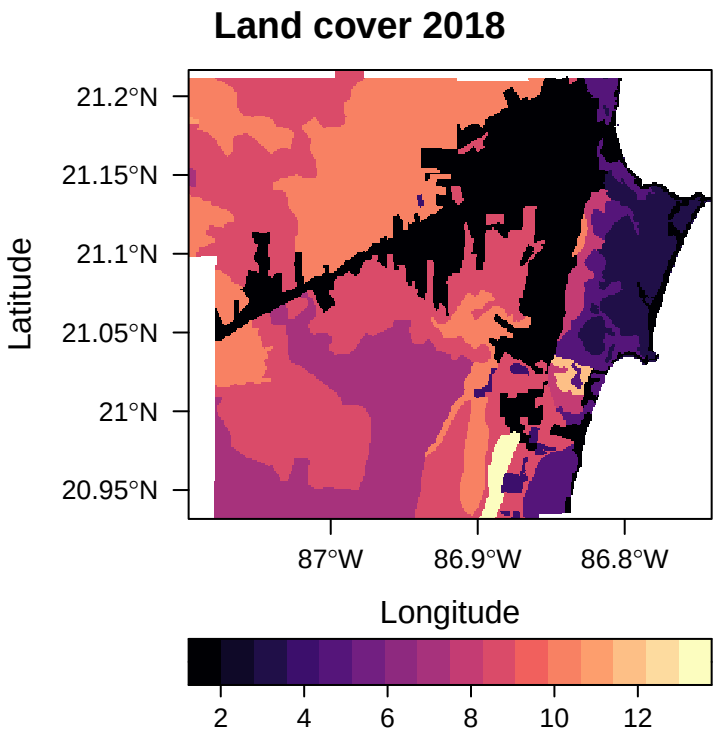
indicator	fclass	value	units
Total population	POP_2025	947982.00	inhabitants
Footprint area	Build_up_2025	300.16	km2
Population density	density	3158.26	inh/km2

2.3.4 Land cover areas

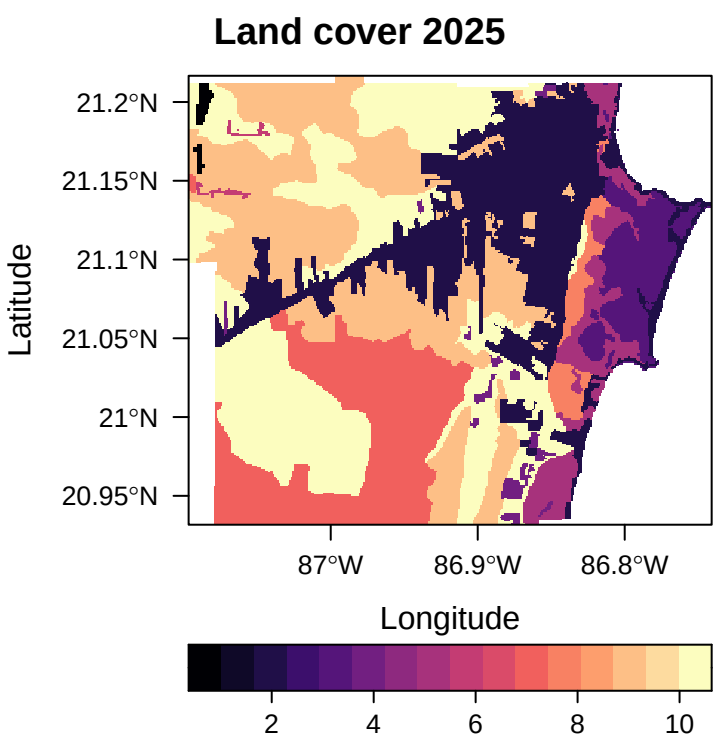
This function calculates areas for each land cover in a specific scenario.

```r
levelplot(land_cover_18,
  margin = FALSE, colorkey = list(space = "bottom"),
```

```
  main = "Land cover 2018"
)
```




```
levelplot(land_cover,
  margin = FALSE, colorkey = list(space = "bottom"),
  main = "Land cover 2025"
)
```



```r
land_l <- stack(land_cover_18, land_cover)
lc_areas <- land_cover_areas(land_l, land.cover.classes)

kable(lc_areas, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

cat	fclass	count	layer	value	units
1	Agricultural land	NA	Land_cover_2018	0.00	km2
2	Urban	25098	Land_cover_2018	231.51	km2
3	Water bodies	6031	Land_cover_2018	55.63	km2
4	Bare land	798	Land_cover_2018	7.36	km2
5	Mangrove	5659	Land_cover_2018	52.20	km2
6	Pastureland	NA	Land_cover_2018	0.00	km2
7	Forest	16227	Land_cover_2018	149.68	km2
8	Forest2	1794	Land_cover_2018	16.55	km2
9	Forest3	34474	Land_cover_2018	318.00	km2
10	Forest4	24906	Land_cover_2018	229.74	km2
1	Agricultural land	390	Land_cover	3.60	km2
2	Urban	25165	Land_cover	232.13	km2
3	Water bodies	6049	Land_cover	55.80	km2
4	Bare land	1203	Land_cover	11.10	km2
5	Mangrove	5639	Land_cover	52.02	km2
6	Pastureland	243	Land_cover	2.24	km2
7	Forest	16789	Land_cover	154.87	km2
8	Forest2	2321	Land_cover	21.41	km2
9	Forest3	28434	Land_cover	262.28	km2
10	Forest4	30220	Land_cover	278.76	km2

2.3.5 Land cover loss

This function calculates the amount of a specific land coverage that is become to a different land cover, between different scenarios. This indicator can be calculate for a specific land coverage loss or different coverages including in the same raster.

```r
land_loss <- land_cover_loss(land_l, areas = lc_areas)

kable(land_loss, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Land cover loss	Agricultural land	-3.60	km2
Land cover loss	Bare land	-3.74	km2
Land cover loss	Forest	-5.19	km2
Land cover loss	Forest2	-4.86	km2
Land cover loss	Forest3	55.72	km2
Land cover loss	Forest4	-49.02	km2
Land cover loss	Mangrove	0.18	km2
Land cover loss	Pastureland	-2.24	km2
Land cover loss	Urban	-0.62	km2
Land cover loss	Water bodies	-0.17	km2

2.3.6 Green land per capita

This function calculates the total green area within the urban area (forests, parks, gardens, etc.) per inhabitant.

```
green_land <- land_cover[[1]]
green_land[green_land != 8] <- NA

green_a_capita <- green_area_pcapita(green_land, pop = population_base)

kable(green_a_capita, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Green area percapita	green area capita	180.68	m2/inh

2.3.7 Roads length

This function calculates the roads kilometers per each fclass in the study area.

```
roads_l <- roads_length(roads.cun)

kable(roads_l, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	units	fclass	value
Roads length	km	primary_road	454.8571
Roads length	km	secondary_road	185.7483

2.3.8 Roads density

This function calculates the roads kilometers per square kilometer in the study area. Road density indicates the level of infrastructure penetration. While high density implies good connectivity, an excessively high density may suggest an over-dominance of car-oriented infrastructure and high impervious surface coverage, which can contribute to heat island effects and runoff issues.

```
roads_dens <- roads_density(roads.cun, buildup_base)

kable(roads_dens, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Roads density	road density	2.13	km/km2

2.3.9 Intersection density

This function calculates the average number of urban crossroads per pixel. Intersection density is a widely recognized proxy for walkability. A higher density typically indicates a finer street grain (smaller blocks) and greater connectivity, which offers pedestrians more route choices and shorter distances between destinations. Conversely, low intersection density often characterizes car-dependent suburban designs.

```r
intersection_dens <- intersection_density(roads.cun, buildup_base)
#> although coordinates are longitude/latitude, st_intersection assumes that they
#> are planar

kable(intersection_dens, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Intersection density	intersection density	4.3	int/km2

2.3.10 Cycle track density

This function calculates the cycle track kilometers per square kilometer of urban area.

This measures the supply of active mobility infrastructure relative to the city size. It helps identify if the investment in cycling networks matches the scale of the urban area.

```r
cycle_dens <- cycle_track_density(cycle.cun, buildup_base)

kable(cycle_dens, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Cycle density	cycle density	0.16	km/km2

2.3.11 Cycle proximity

This function calculates the total and the percentage of the population that lives within a maximum recommended distance from a cycle track. This indicator reveals who actually has access to the network, highlighting gaps where neighborhoods are disconnected from safe cycling routes.

```r
cycle_prox <- cycle_proximity(cycle.cun, population_base)

kable(cycle_prox, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Cycle proximity	cycle	158592.00	inhabitants
Cycle proximity	cycle	16.73	%

2.3.12 Public transport proximity

This function calculates the total and the percentage of the population that lives within a maximum recommended distance from a public transport station, according to the transport classification.

High proximity ensures that mass transit is a viable option for residents, reducing dependency on private vehicles and lowering carbon emissions.

```r
ptransport_prox <- public_transport_proximity(transport.cun,
  pop = population_base
)
#> Warning: st_centroid assumes attributes are constant over geometries
```

```r
#> st_as_s2(): dropping Z and/or M coordinate

kable(ptransport_prox, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Public transport proximity	public transport	39044.00	inhabitants
Public transport proximity	public transport	4.12	%

2.3.13 Amenities proximity

This function calculates the total and the percentage of the population that lives within the maximum recommended distance to each of the following urban public services and amenities. Setting save = FALSE prevents the function from writing intermediate shapefiles to your local working directory.

```r
amenities_prox <- amenities_proximity(amenities.cun,
  pop = population_base,
  save = FALSE
)

kable(amenities_prox, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Amenities proximity	grocery_store	148009.00	inhabitants
Amenities proximity	grocery_store	15.61	%
Amenities proximity	school	499932.00	inhabitants
Amenities proximity	school	52.74	%
Amenities proximity	health	717579.00	inhabitants
Amenities proximity	health	75.70	%

2.3.14 Jobs proximity

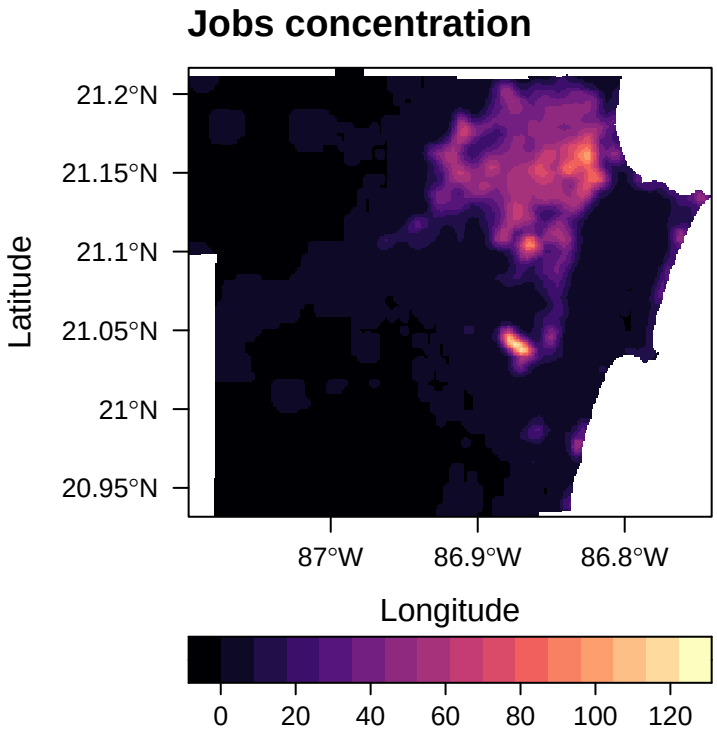
This function calculates the percentage of the population that lives within the areas with high job density in the urban area. This indicator helps in evaluation of the proportion of residents living near major employment clusters or high job-density zones.

```r
jobs_prox <- jobs_proximity(jobs_distribution[[1]], pop = population_base)

kable(jobs_prox, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Jobs proximity	jobs	884202.00	inhabitants
Jobs proximity	jobs	93.27	%

```r
levelplot(jobs_distribution,
  margin = FALSE, colorkey = list(space = "bottom"),
  main = "Jobs concentration"
)
```



2.3.15 Population in risk

This function calculates the number and percentage of the population that is exposed to hazards because they live within the zone of influence of natural and anthropogenic sources.

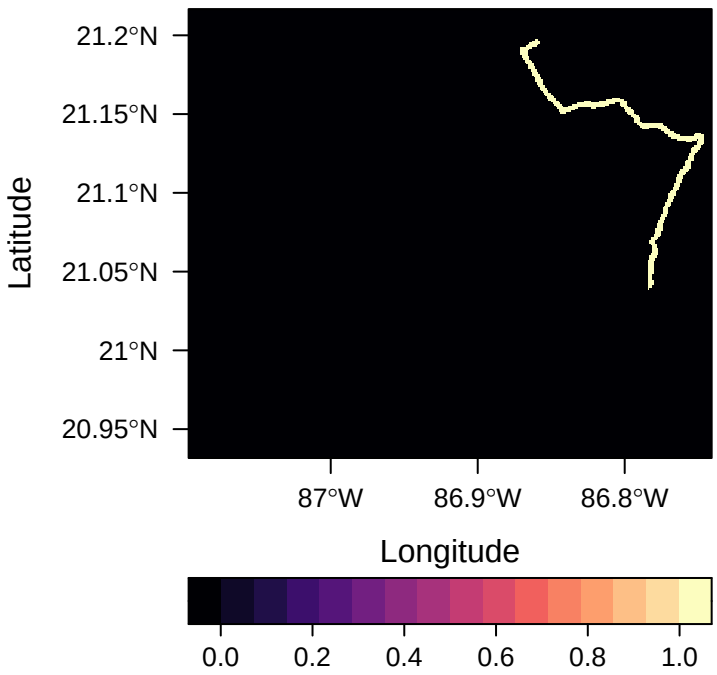
It quantifies the human risk associated with current settlement patterns and helps prioritize areas for mitigation strategies or relocation policies.

```
pop_in_hazard <- hazard_exposure(hazard_zones, pop = population_base)

kable(pop_in_hazard, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Population exposed to risk	Hazard	40964.00	inhabitants
Population exposed to risk	Hazard	4.32	%

```
levelplot(hazard_zones, margin = FALSE, colorkey = list(space = "bottom"))
```



2.3.16 Land consumption

This function calculates land consumption between two urban growth scenarios. Land consumption is the amount of land predicted to change from natural habitats or agricultural uses into urban human settlements between the base year and the horizon year.

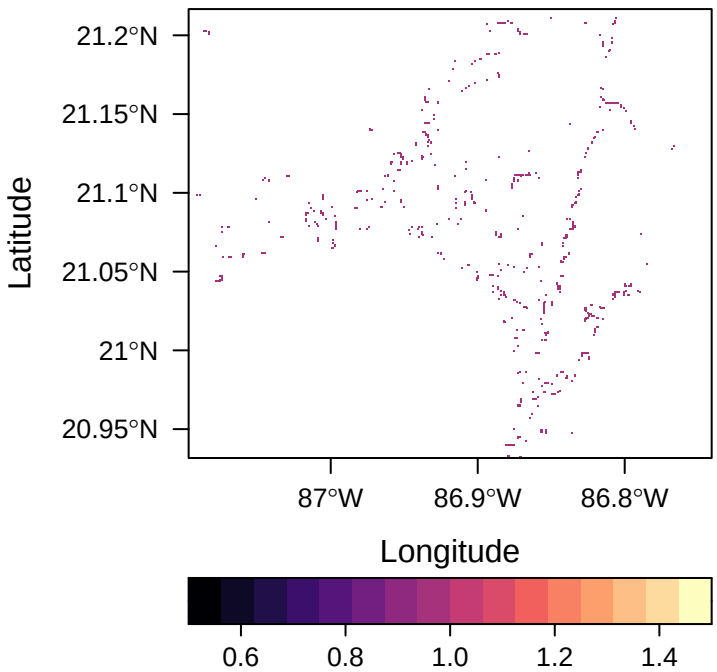
```
l_consumption <- land_consumption(buildup_base, buildup_horizon)

kable(l_consumption, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Land consumption	Build_up_2030 - Build_up_2025	7.79	km2

Note: The ***land_consumption*** *function automatically generates a raster object in your global environment named* ***urban.expansion_ Build_up_2030*** *representing the expanded areas. You can plot this object directly:*

```
levelplot(`urban.expansion_ Build_up_2030`,
  margin = FALSE,
  colorkey = list(space = "bottom")
)
```



2.3.17 Infill

This function calculates the amount of land that was located within the boundaries of the urban area in the base scenario and that at least doubled its population between the base year and the horizon year. It is also known as intensification area.

```
infill_area <- infill(population_base, population_horizon)

kable(infill_area, "latex", booktabs = TRUE) |>
  kable_styling(latex_options = "HOLD_position")
```

indicator	fclass	value	units
Infill	infill	225.3	km2

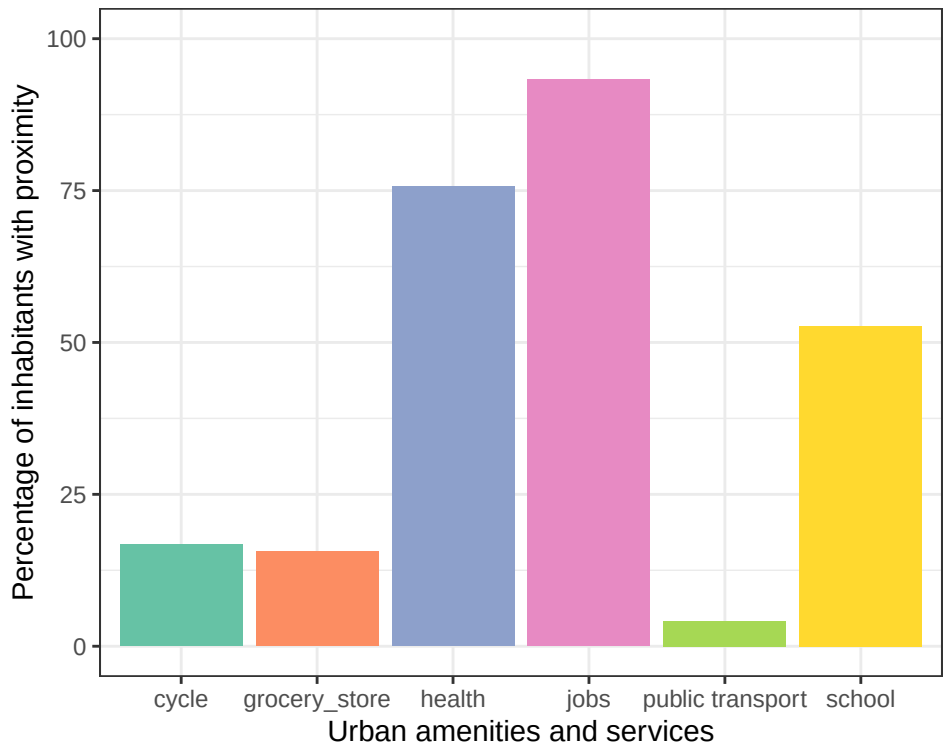
2.3.18 Results analysis

Select proximity data for visualization

```
## Join all proximities in a data frame
proximities <- rbind(cycle_prox, ptransport_prox, amenities_prox, jobs_prox)

## Select only percentages
proximities_pct <- subset(proximities, units == "%")
```

Visualize results

```
## library for plotting proximity results
library(ggplot2)
p <- ggplot(proximities_pct, aes(x = fclass, y = value, fill = fclass)) +
  geom_bar(stat = "identity") +
  scale_fill_brewer(palette = "Set2") +
  theme_bw() +
  ylab("Percentage of inhabitants with proximity") +
```

```r
  xlab("Urban amenities and services") +
  theme(legend.position = "none") +
  ylim(0, 100)

p
```